

# Resolución del Práctico 2

## Testing de Software 2026

Tomás Rodeghiero

### Índice

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>1. Ejercicio 1</b>                                    | <b>3</b> |
| <b>2. Ejercicio 2</b>                                    | <b>3</b> |
| 2.1. Consigna . . . . .                                  | 3        |
| 2.2. Estrategia de parametrización . . . . .             | 3        |
| 2.3. Casos elegidos . . . . .                            | 4        |
| 2.4. Suite . . . . .                                     | 4        |
| 2.5. Resultado sobre la versión defectuosa . . . . .     | 5        |
| 2.6. Cómo ejecutar . . . . .                             | 5        |
| <b>3. Ejercicio 3</b>                                    | <b>5</b> |
| 3.1. Consigna . . . . .                                  | 5        |
| 3.2. Modelo de la clase . . . . .                        | 5        |
| 3.3. Implementación de <code>repOk</code> . . . . .      | 6        |
| 3.4. Correcciones sobre <code>StackAr</code> . . . . .   | 6        |
| 3.5. Suite de tests . . . . .                            | 7        |
| 3.6. Tests de <code>repOk</code> con reflexión . . . . . | 7        |
| 3.7. Inconvenientes encontrados . . . . .                | 8        |
| 3.8. Cómo ejecutar . . . . .                             | 8        |
| <b>4. Ejercicio 4</b>                                    | <b>8</b> |
| 4.1. Consigna . . . . .                                  | 8        |
| 4.2. Estructura de los CSV . . . . .                     | 9        |
| 4.3. Mapeo CSV $\rightarrow$ test . . . . .              | 9        |
| 4.4. Suite . . . . .                                     | 9        |
| 4.5. Tests negativos adicionales . . . . .               | 10       |
| 4.6. Cómo ejecutar . . . . .                             | 10       |

|                                             |           |
|---------------------------------------------|-----------|
| <b>5. Ejercicio 5</b>                       | <b>10</b> |
| 5.1. Consigna . . . . .                     | 10        |
| 5.2. Contexto . . . . .                     | 10        |
| 5.3. Versión original . . . . .             | 11        |
| 5.4. Tests . . . . .                        | 11        |
| 5.5. Fallas detectadas . . . . .            | 12        |
| 5.6. Corrección . . . . .                   | 12        |
| 5.7. Cómo ejecutar . . . . .                | 13        |
| <b>6. Ejercicio 6</b>                       | <b>13</b> |
| 6.1. Consigna . . . . .                     | 13        |
| 6.2. Modelo de la clase . . . . .           | 13        |
| 6.3. Invariante de representación . . . . . | 13        |
| 6.4. Implementación . . . . .               | 13        |
| 6.5. Tests parametrizados . . . . .         | 14        |
| 6.6. Cómo ejecutar . . . . .                | 15        |

# 1. Ejercicio 1

El primer ejercicio es la lectura del capítulo 3 de *Introduction to Software Testing* (Ammann y Offutt, 2.<sup>a</sup> edición), *Test Automation*.

Los conceptos más importantes son:

- **Test Requirement (TR) y criterio de cobertura (TCC).** El TR es una propiedad sintáctica que un test tiene que cumplir; el TCC es la regla que, dado el artefacto bajo prueba, genera el conjunto de TRs.
- **Subsumption.**  $C_1$  subsume a  $C_2$  si todo test set que cubre  $C_1$  también cubre  $C_2$ . Ordena criterios por fuerza.
- **Patrón Arrange–Act–Assert.** Preparar el escenario, ejecutar la operación, verificar el resultado. Se aplica en los ejercicios 3 y 6.
- **Test-Driven Development.** Escribir el test antes de la implementación.
- **Model-Driven Test Design (MDTD).** Marco que aplica los mismos criterios sobre cuatro modelos (control flow, data flow, logic, input space).

# 2. Ejercicio 2

## 2.1. Consigna

Pasar los tests del ejercicio 3 del TP1 a tests parametrizados sobre `assignment2_exercises.SimpleRoutines`. La consigna pide al menos un `@MethodSource` y al menos un caso en CSV (`@CsvSource` o `@CsvFileSource`).

## 2.2. Estrategia de parametrización

Asigné una fuente distinta a cada método para cubrir las tres alternativas que pide la consigna:

- `findLast`: `@CsvSource inline`.
- `lastZero` y `countPositive`: `@MethodSource`, porque conviene devolver `int[]` directamente.
- `oddOrPos`: `@CsvFileSource`, leyendo `odd_or_pos_cases.csv` desde `src/test/resources`.

Para los tipos no primitivos (`int[]`), `@MethodSource` devolviendo `Stream<Arguments>` es la forma estándar en JUnit 5.

## 2.3. Casos elegidos

Reutilicé los casos (c) y (d) del análisis RIPR del TP1. (c) ejecuta el defecto pero no produce falla; (d) lo ejecuta y sí produce falla.

| Programa      | Caso (c) (pasa) | Caso (d) (revela el defecto) |
|---------------|-----------------|------------------------------|
| findLast      | [5,2,3], 2 ->1  | [2,3,5], 2 ->0               |
| lastZero      | [1,0,2] ->1     | [0,1,0] ->2                  |
| countPositive | [-4,2,2] ->2    | [-4,2,0,2] ->2               |
| oddOrPos      | [2,4,-2] ->2    | [-3,-2,0,1,4] ->3            |

Para countPositive y oddOrPos agregué también el caso (b) (arreglo vacío), donde el defecto ni se alcanza.

## 2.4. Suite

```
1 @ParameterizedTest(name = "findLast({0}, {1}) -> {2}")
2 @CsvSource({
3     "'5,2,3', 2, 1",
4     "'2,3,5', 2, 0"
5 })
6 void findLast_casosDelEjercicio3(String valores, int objetivo,
7     int esperado) {
8     assertEquals(esperado, SimpleRoutines.findLast(parseArray(
9         valores), objetivo));
10 }
11
12 @ParameterizedTest(name = "lastZero({0}) -> {1}")
13 @MethodSource("lastZeroCases")
14 void lastZero_casosDelEjercicio3(int[] entrada, int esperado) {
15     assertEquals(esperado, SimpleRoutines.lastZero(entrada));
16 }
17
18 @ParameterizedTest(name = "oddOrPos({0}) -> {1}")
19 @CsvFileSource(resources = "/assignment2_exercises/
20     odd_or_pos_cases.csv", numLinesToSkip = 1)
21 void oddOrPos_casosDelEjercicio3(String valores, int esperado) {
22     assertEquals(esperado, SimpleRoutines.oddOrPos(parseArray(
23         valores)));
24 }
```

## 2.5. Resultado sobre la versión defectuosa

La `SimpleRoutines` de este TP es la versión *con* los cuatro defectos del TP1. Cuando corro la suite parametrizada me salen exactamente los cuatro *failures* esperados, uno por programa, correspondientes a los casos (d):

- `findLast([2,3,5], 2)` devuelve `-1` en lugar de `0` (la guarda `i > 0` omite el índice cero).
- `lastZero([0,1,0])` devuelve `0` en lugar de `2` (recorre de izquierda a derecha y se queda con el primer cero).
- `countPositive([-4,2,0,2])` devuelve `3` en lugar de `2` (la condición `>= 0` cuenta el cero como positivo).
- `oddOrPos([-3,-2,0,1,4])` devuelve `2` en lugar de `3` (la condición `% 2 == 1` no detecta impares negativos).

La suite parametrizada conserva el mismo poder de detección que el conjunto no parametrizado del TP1.

## 2.6. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true  
-Dtest=SimpleRoutinesParameterizedTest test
```

# 3. Ejercicio 3

## 3.1. Consigna

Crear una suite completa para `StackAr`, con fixture compartido vía `@BeforeEach`, AAA explícito en cada test, al menos dos tests negativos, e implementación de `repOk()` con el invariante de la consigna.

## 3.2. Modelo de la clase

`StackAr` es una pila acotada sobre un arreglo `elems` con un puntero `sp` al tope, donde:

- `sp = -1` representa la pila vacía;
- las posiciones `0, 1, ..., sp` contienen elementos válidos;
- las posiciones `sp + 1, ..., elems.length - 1` están libres (`null`).

### 3.3. Implementación de repOk

```
1 public boolean repOk() {
2     if (elems == null) {
3         return false;
4     }
5     if (sp < -1 || sp >= elems.length) {
6         return false;
7     }
8     for (int i = 0; i < elems.length; i++) {
9         if (i > sp && elems[i] != null) {
10             return false;
11         }
12         if (i <= sp && elems[i] == null) {
13             return false;
14         }
15     }
16     return true;
17 }
```

### 3.4. Correcciones sobre StackAr

Implementar el invariante me hizo encontrar dos problemas en el código provisto:

1. **pop() no respetaba LIFO.** La versión original devolvía el nuevo tope (es decir, lo que quedaba después de decrementar `sp`) y no limpiaba la celda. Esto viola la cláusula “ $\forall i > sp : \text{elems}[i] = \text{null}$ ”. La versión corregida guarda el tope, lo limpia y decrementa:

```
1 public Object pop() throws IllegalStateException {
2     if (isEmpty()) {
3         throw new IllegalStateException();
4     }
5     Object rv = this.elems[sp];
6     this.elems[sp] = null;
7     sp--;
8     return rv;
9 }
```

2. **push aceptaba null.** La cláusula “ $\forall i \leq sp : \text{elems}[i] \neq \text{null}$ ” es incompatible con eso. Agregué una validación que rechaza null con `IllegalArgumentException`.

### 3.5. Suite de tests

Cubrí todos los métodos públicos. Cada test marca las tres fases con comentarios. Algunos ejemplos representativos:

```
1 @BeforeEach
2 void setUp() {
3     // Todas las pruebas parten de una pila acotada de capacidad
4     // 3.
5     stack = new StackAr(3);
6 }
7 @Test
8 void popDebeRetornarUltimoElementoYReducirSize() {
9     // arrange
10    stack.push("A");
11    stack.push("B");
12
13    // act
14    Object valor = stack.pop();
15
16    // assert
17    assertEquals("B", valor);
18    assertEquals(1, stack.size());
19    assertEquals("A", stack.top());
20 }
21
22 @Test
23 void pushConNull_debeLanzarExcepcion() {
24    assertThrows(IllegalArgumentException.class, () -> stack.push
25        (null));
26 }
```

Los tests negativos cubren cuatro escenarios: `push` en pila llena, `push(null)`, `pop` en pila vacía y `top` en pila vacía.

### 3.6. Tests de `rep0k` con reflexión

Para probar las ramas `false` de `rep0k` no alcanza la API pública: la clase no permite dejar `sp` con un valor imposible o un hueco en la zona activa. Usé `java.lang.reflect.Field` para corromper el estado interno de forma controlada. El libro discute esta técnica al hablar de *controllability*: cuando el código bajo prueba esconde su estado, hace falta un

mecanismo adicional para forzar las situaciones que se quieren verificar.

```
1 @Test
2 void repOkConHuecoEnZonaActiva_debeRetornarFalse() throws
    Exception {
3     // arrange: dejamos un null dentro de la zona "activa" (0..sp
        ).
4     stack.push("A");
5     stack.push("B");
6     Object[] elems = getElems(stack);
7     elems[1] = null;
8
9     // act
10    boolean ok = stack.repOk();
11
12    // assert
13    assertFalse(ok);
14 }
```

### 3.7. Inconvenientes encontrados

- No se puede romper el invariante desde la API pública: para algunos casos negativos hizo falta reflexión.
- Implementar `repOk` expuso el problema de `pop`, que no era visible mientras los tests solo miraban el resultado del método aislado.
- `equals` y `hashCode` usan todo el arreglo, no solo la zona activa. Dos pilas con misma capacidad y mismos elementos son iguales; con distinta capacidad y mismos elementos no. Es coherente con el modelo de pila acotada, pero conviene tenerlo presente al diseñar tests.

### 3.8. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=StackArTest test
```

## 4. Ejercicio 4

### 4.1. Consigna

Crear tests parametrizados en CSV para `Min.min(List<? extends T>)`. Agregar tests negativos para las tres excepciones documentadas: `IllegalArgumentException`,



NullPointerException y ClassCastException.

## 4.2. Estructura de los CSV

Codifiqué cada lista como un único string con elementos separados por ;. Una sola columna “valores” por caso, el CSV queda compacto y legible:

```
# min_valid_cases.csv
valores,esperado
"3;2;1",1
"5",5
"-2;4;0",-2
"7;7;9",7
"10;-1;3;-1",-1
```

```
# min_invalid_cases.csv
valores,excepcion
"EMPTY",IllegalArgumentException
"null;2;3",NullPointerException
"1;null;3",NullPointerException
```

## 4.3. Mapeo CSV → test

- EMPTY representa una lista vacía.
- Un token null dentro de la lista se traduce a un null en la List<Integer>.
- El nombre de la excepción esperada se mapea a Class<? extends Throwable> en el propio test, así el CSV se mantiene libre de detalles del lenguaje.

## 4.4. Suite

```
1 @ParameterizedTest(name = "min({0}) -> {1}")
2 @CsvFileSource(resources = "/assignment2_exercises/
   min_valid_cases.csv", numLinesToSkip = 1)
3 void min_conCasosValidosCsv_debeRetornarMinimo(String valores,
   int esperado) {
4     List<Integer> lista = parseIntegerList(valores);
5     assertEquals(esperado, Min.min(lista));
6 }
7
8 @ParameterizedTest(name = "min({0}) lanza {1}")
```

```

9  @CsvFileSource(resources = "/assignment2_exercises/
    min_invalid_cases.csv", numLinesToSkip = 1)
10 void min_conCasosInvalidosCsv_debeLanzarExcepcion(String valores,
    String excepcionEsperada) {
11     List<Integer> lista = parseIntegerList(valores);
12     assertThrows(exceptionType(excepcionEsperada), () -> Min.min(
        lista));
13 }

```

## 4.5. Tests negativos adicionales

Dos escenarios no se prestan a CSV y van como `@Test` clásicos:

- Lista null (que no es lo mismo que una lista vacía).
- Lista heterogénea con elementos no mutuamente comparables. Hay que usar *raw type* para inyectar `[1, "dos"]` sin que el compilador rechace el código:

```

1  @Test
2  @SuppressWarnings({"rawtypes", "unchecked"})
3  void min_conElementosNoComparables_debeLanzarClassCastException()
    {
4      List lista = Arrays.asList(1, "dos");
5      assertThrows(ClassCastException.class, () -> Min.min(lista));
6  }

```

## 4.6. Cómo ejecutar

```

mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true
    -Dtest=MinCsvParameterizedTest test

```

# 5. Ejercicio 5

## 5.1. Consigna

Crear tests parametrizados para `ZuneBug.currentYear(int days)`. Si los tests revelan alguna falla, hacer debugging y corregir el defecto.

## 5.2. Contexto

ZuneBug reproduce un bug real del firmware del Zune de Microsoft: el 31 de diciembre de 2008 (año bisiesto) todos los dispositivos colgaron al intentar determinar el año actual a

partir de los días transcurridos desde 1980. La rutina entraba en un loop infinito por una lógica inconsistente al tratar bisiestos.

### 5.3. Versión original

```
1 public static int currentYear(int days) {
2     int year = 1980;
3     while (days > 365) {
4         if (isLeapYear(year)) {
5             if (days > 366) {
6                 days -= 366;
7                 year += 1;
8             }
9             // Sin else: si la condicion no se cumple, ni days ni
10             year
11             // cambian, entonces el bucle no progresa -> loop
12             infinito.
13         } else {
14             days -= 365;
15             year += 1;
16         }
17     }
18     return year;
19 }
```

### 5.4. Tests

Armé dos tests parametrizados complementarios:

1. `casosDeBorde` con `@MethodSource`, valores específicos elegidos por análisis de bordes: 1, 365, 366, 367, 731, 1461.
2. `currentYear_debeCoincidirConOracle` con `@ValueSource`, comparación diferencial contra `ZuneBug.oracle`, que usa `LocalDate.plusDays`. Es un caso pequeño pero claro de *test oracle*.

Los dos tests envuelven la llamada en `assertTimeoutPreemptively(Duration.ofMillis(200))`, así un eventual loop infinito se traduce en un *failure* explícito y no en un *hang* que bloquea la build.

```
1 @ParameterizedTest(name = "days={0} coincide con oracle")
2 @ValueSource(ints = {1, 2, 30, 31, 59, 60, 100, 365, 366, 367,
3                     730, 731, 1095, 1461, 10000})
```

```

4 void currentYear_debeCoincidirConOracle(int days) {
5     assertTimeoutPreemptively(Duration.ofMillis(200), () -> {
6         int esperado = ZuneBug.oracle(days);
7         int actual = ZuneBug.currentYear(days);
8         assertEquals(esperado, actual);
9     });
10 }

```

## 5.5. Fallas detectadas

- **Loop infinito** cuando `days = 366` en un año bisiesto: la guarda exterior `days > 365` se cumple, pero la rama bisiesta solo descuenta si `days > 366`. Sin `else`, el cuerpo del `while` no progresa.
- **Off-by-one** en transiciones exactas: por ejemplo, con `days = 731` la versión original devolvía un año menos que el oráculo.

## 5.6. Corrección

Reescribí el bucle para que la condición dependa de cuántos días tiene el año en curso:

```

1 public static int currentYear(int days) {
2     int year = 1980;
3     while (true) {
4         int daysInYear = isLeapYear(year) ? 366 : 365;
5         if (days >= daysInYear) {
6             days -= daysInYear;
7             year += 1;
8         } else {
9             break;
10        }
11    }
12    return year;
13 }

```

Mientras quedan días suficientes para completar el año en curso (`>= daysInYear`), se descuentan y se avanza; si no, se corta y se devuelve el año. Así el bucle siempre progresa (no hay loop infinito) y la comparación es `>=`, que arregla el off-by-one.

## 5.7. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true  
-Dtest=ZuneBugParameterizedTest test
```

Después de la corrección: Tests run: 21, Failures: 0, Errors: 0.

## 6. Ejercicio 6

### 6.1. Consigna

Implementar `repOK()` para `BoundedQueue` y verificar con tests parametrizados que el invariante se mantiene tras secuencias de `enQueue` y `deQueue`.

### 6.2. Modelo de la clase

`BoundedQueue` es una cola FIFO acotada implementada como buffer circular. Los atributos son `elements` (arreglo), `size`, `front` (índice del próximo a desencolar), `back` (índice donde se escribe el próximo `enQueue`) y `capacity`.

### 6.3. Invariante de representación

`repOK` chequea:

1. `elements != null` y `elements.length == capacity`.
2. `capacity >= 0`.
3.  $0 \leq \text{size} \leq \text{capacity}$ .
4. Caso `capacity == 0`: `size`, `front` y `back` valen cero.
5.  $0 \leq \text{front} < \text{capacity}$  y  $0 \leq \text{back} < \text{capacity}$ .
6. **Buffer circular**: `back == (front + size) % capacity`.
7. Zona ocupada sin huecos:  $\forall i \in [0, \text{size})$ , `elements[(front+i) % capacity] != null`.
8. Zona libre limpia: el resto de las posiciones son `null`.

### 6.4. Implementación

```
1 public boolean repOK() {  
2     if (elements == null) return false;  
3     if (capacity < 0 || elements.length != capacity) return false  
    ;  
}
```

```

4     if (size < 0 || size > capacity) return false;
5     if (capacity == 0) return size == 0 && front == 0 && back ==
      0;
6     if (front < 0 || front >= capacity || back < 0 || back >=
      capacity) return false;
7     if (back != (front + size) % capacity) return false;
8     boolean[] occupied = new boolean[capacity];
9     for (int i = 0; i < size; i++) {
10        int idx = (front + i) % capacity;
11        occupied[idx] = true;
12        if (elements[idx] == null) return false;
13    }
14    for (int i = 0; i < capacity; i++) {
15        if (!occupied[i] && elements[i] != null) return false;
16    }
17    return true;
18 }

```

## 6.5. Tests parametrizados

Para no escribir un test por secuencia armé un mini-DSL textual: `E:valor` para `enqueue(valor)` y `D` para `dequeue()`. Una cadena como `.E:a;E:b;D;E:c` encadena las operaciones, y un método `apply` las ejecuta una por una, chequeando `repOK` en cada paso.

### Escenarios válidos (@MethodSource)

Cubren situaciones representativas: llenar y vaciar, *wrap-around* (cuando `back` da la vuelta al final del arreglo), capacidades chicas (1 y 2) donde la aritmética modular es más delicada.

| Capacidad | Operaciones             | Cola final |
|-----------|-------------------------|------------|
| 3         | E:a;E:b;D;E:c           | [b, c]     |
| 3         | E:x;E:y;D;E:z;E:w       | [y, z, w]  |
| 4         | E:1;E:2;E:3;D;D;E:4;E:5 | [3, 4, 5]  |
| 2         | E:p;E;q;D;E:r           | [q, r]     |
| 1         | E:solo;D                | []         |

### Escenarios inválidos

Las operaciones inválidas (`dequeue` en vacía, `enqueue(null)`, `enqueue` en llena) deben lanzar la excepción documentada y **además el invariante debe seguir cumpliéndose después del error**: una operación que falla no puede corromper la cola.

```

1 @ParameterizedTest(name = "cap={0}, setup={1}, fail={2}, ex={3}")
2 @MethodSource("invalidScenarios")
3 void repOK_debeMantenerseCuandoUnaOperacionInvalidaLanzaExcepcion
4     (
5         int capacity,
6         String setupOperations,
7         String failingOperation,
8         Class<? extends Throwable> expectedException
9     ) {
10     BoundedQueue queue = new BoundedQueue(capacity);
11     for (String op : splitOperations(setupOperations)) {
12         apply(queue, op, new ArrayList<>());
13         assertTrue(queue.repOK());
14     }
15     assertThrows(expectedException, () -> apply(queue,
16         failingOperation, new ArrayList<>()));
17     assertTrue(queue.repOK());
18 }

```

## 6.6. Cómo ejecutar

```

mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true
-Dtest=BoundedQueueParameterizedTest test

```